8 Reactive Spring: Resilience and scalability

This chapter covers

- Understanding reactive programming with Reactor and Spring
- Building reactive servers with Spring WebFlux and Spring Data R2DBC
- Building reactive clients with WebClient
- Improving resilience for applications with Reactor
- Testing reactive applications with Spring and Testcontainers

Polarsophia, the organization behind the Polar Bookshop business, is very happy with the progress of its new software product. Its mission is to spread knowledge and awareness about the North Pole and the Arctic, and making its book catalog available worldwide is an essential part of this.

The Catalog Service application you built so far is a good starting point. It fulfills the requirements of browsing and managing books, and it does that while following cloud native patterns and practices. It's self-contained and stateless. It uses a database as a backing service to store the state. It can be configured externally through environment variables or a configuration server. It respects environment parity. It's verified through the automated execution of tests as part of a deployment pipeline, following continuous delivery practices. For maximum portability, it's also containerized and can be deployed to a Kubernetes cluster using native functionality like service discovery, load balancing, and replication.